

THE PROBLEM

Enterprises accumulate dozens of security & compliance policies written by different teams over many years, so contradictions ("rotate passwords every 90 days" vs "never rotate — enforce MFA"), silent redundancy, and stale references to dead tech (TLS 1.0, SHA-1, Server 2012) pile up unnoticed.

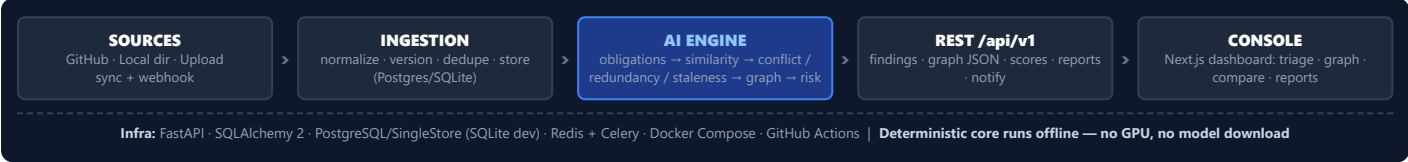
Today those gaps are found **by auditors, not before them** — creating real audit, legal, and breach exposure.

THE SOLUTION

Sentinal continuously ingests policies (GitHub, folders, upload), extracts every **obligation**, and runs a deterministic AI engine that flags conflicts, redundancy, and staleness — each finding cites the **exact triggering text** and a suggested resolution.

A live operations console ranks findings by severity × confidence, maps them to **ISO 27001 / NIST / GDPR / COBIT**, and collapses 20+ hrs/quarter of manual reconciliation into a ranked, audit-ready list.

ARCHITECTURE



SUCCESS METRICS & ACCEPTANCE (SRS §6)

METRIC	TARGET	ACCEPTANCE TEST
Conflict detection rate	> 75%	tests/ai/test_conflicts.py
Redundancy detection	> 70%	tests/ai/test_redundancy.py
Staleness detection	> 90%	tests/ai/test_staleness.py
False-positive rate	< 20%	tests/ai/test_precision.py
Obligation-extraction accuracy	> 80%	tests/ai/test_obligations.py
End-to-end run	passes	POST /api/v1/analysis/run

WHY IT WINS

- Explainable** every finding cites source span + fix — not a black box.
- Precision-first** FPR target <20%; downgrades severity over crying wolf.
- Continuous** webhook re-analysis on every policy change, not a one-off scan.
- Real & live** deployed full-stack; deterministic core needs no ML deps.

LINKS

LIVE DEMO
sentinal-taupe.vercel.app

SOURCE REPO
github.com/rithvin-us/Policy-Conflict-and-Staleness-Detector

LIVE API + OPENAPI
policy-guardian-backend-wbge.onrender.com/docs

DEMO VIDEO / DECK
▶ paste hosted URL here

What is actually "AI" here vs. plain rules?

The core is a **deterministic NLP engine**: modal-verb→strength, verb→action, negation→polarity, topic/scope maps, regex parameter capture, then cross-obligation reasoning over 5 conflict classes (direct / temporal / scope / strength / parameter). An **optional semantic layer** (sentence-transformers / FAISS) only *adds* candidate pairs — it never overturns a deterministic finding. That's a deliberate design choice for precision + reproducibility, not a limitation.

How is this different from a linter or grep?

grep finds strings; Sentinal reasons over **meaning**. It knows "rotate every 90 days" and "never rotate" conflict on the same topic despite zero shared keywords, weighs scope exceptions to avoid false alarms, ranks by severity × confidence, and maps each hit to a compliance clause. A linter can't tell you your MFA policy contradicts your password policy.

Does it scale beyond the seed data?

Connectors ingest arbitrary GitHub repos / folders; ingestion versions + dedupes by content hash; webhooks re-analyze only what changed. NFR-3 target: a 30-policy corpus analyzed in **< 10 s** on the deterministic path, commodity hardware.

END-TO-END FLOW

Configure source → **Sync** (normalize + version) → **Analyze** (obligations → similarity → detectors → graph → scoring) → **Persist** findings + score → **Serve** console → **Report** (MD/HTML/JSON) → **Notify** compliance managers on new HIGH → **Re-trigger** on webhook. Same pipeline whether triggered by schedule or GitHub push.

COMPLIANCE COVERAGE

Every finding carries a **compliance_impact** mapping to **ISO/IEC 27001:2022** §5.1/§5.2, **NIST SP 800-53** (PL-1, PM-1, IA-5), **NIST 800-63B**, **GDPR** Arts. 5/17/24, and **COBIT 2019** APO01 — so an auditor sees not just "these two policies conflict" but "...and here's the control they both touch."

What is real vs. stubbed?

Real & runnable: ingestion, obligation extraction, all 5 conflict detectors, redundancy, staleness (review-age / deprecated-tech / superseded-standard / orphan-owner), risk & governance scoring, policy + obligation knowledge graph, explainable findings, MD/HTML/JSON reports, GitHub + Local + Upload connectors, webhook-in, compliance mapping, dashboard, Docker + CI + tests. **Registered stubs (same interface)**: GitLab, Bitbucket, Drive, OneDrive, SharePoint, Slack, Teams. **Roadmap**: PDF/DOCX ingest, version-diff, NL query, RBAC/SSO.

What's the business case?

Compliance teams burn **20+ hours/quarter** manually reconciling policies for ISO 27001 §5.1/§5.2 audits. Sentinal turns that into reviewing a ranked, evidenced findings list and exports a Compliance Coverage report as audit evidence. The measurable win is **audit-overhead reduction** plus reduced legal/breach exposure from contradictory controls.

TECH STACK

Backend - Python 3.11+ (val. 3.14) - FastAPI · SQLAlchemy 2 - Redis + Celery	AI - Pure-Python deterministic core - opt. sbert / spaCy / FAISS
Frontend - Next.js 14 · TypeScript - Tailwind · React Flow · Recharts	Data / Ops - PostgreSQL / SingleStore - Docker Compose · GH Actions

RELIABILITY BY DESIGN

Degrades gracefully: **no Postgres** → SQLite file; **no Redis/Celery** → in-process sync; **no ML libs** → lexical similarity core. The system is fully functional with **none** of the optional dependencies — **`docker compose up`** brings up the whole stack. Structured JSON logs, **/health** + **/ready** probes, request IDs on every response.